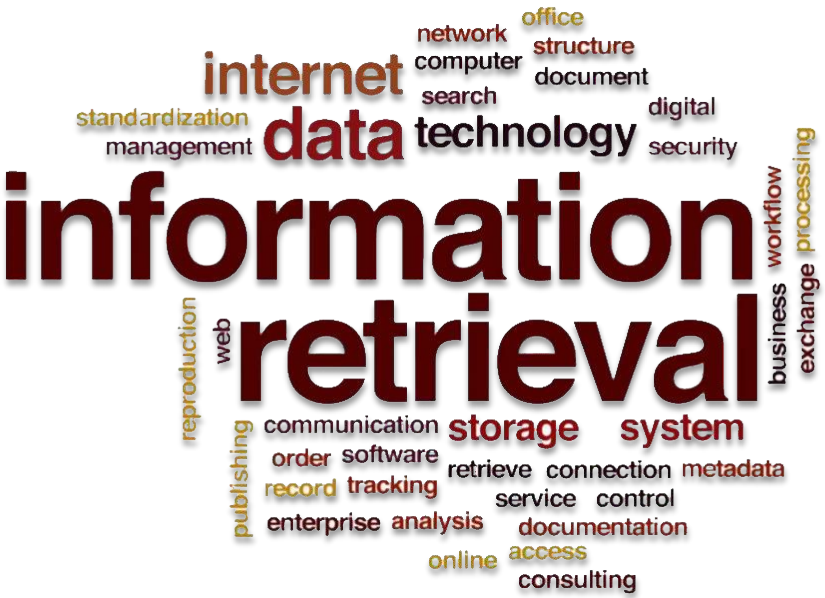

office
network
structure
internet
computer
document
search
digital
standardization
data
technology
management
security
information
workflow
processing
retrieval
business
exchange
reproduction
web
communication
storage
system
order
software
retrieve
connection
metadata
publishing
record
tracking
service
control
enterprise
analysis
documentation
online
access
consulting

What is Information Retrieval ?

Information Retrieval (IR) is concerned with **Searching**, **Representing** and **Manipulating** large collection of electronic text and other human-language data

Information Retrieval (IR) is finding material (usually documents) of an unstructured nature (usually text)

Basic Terms

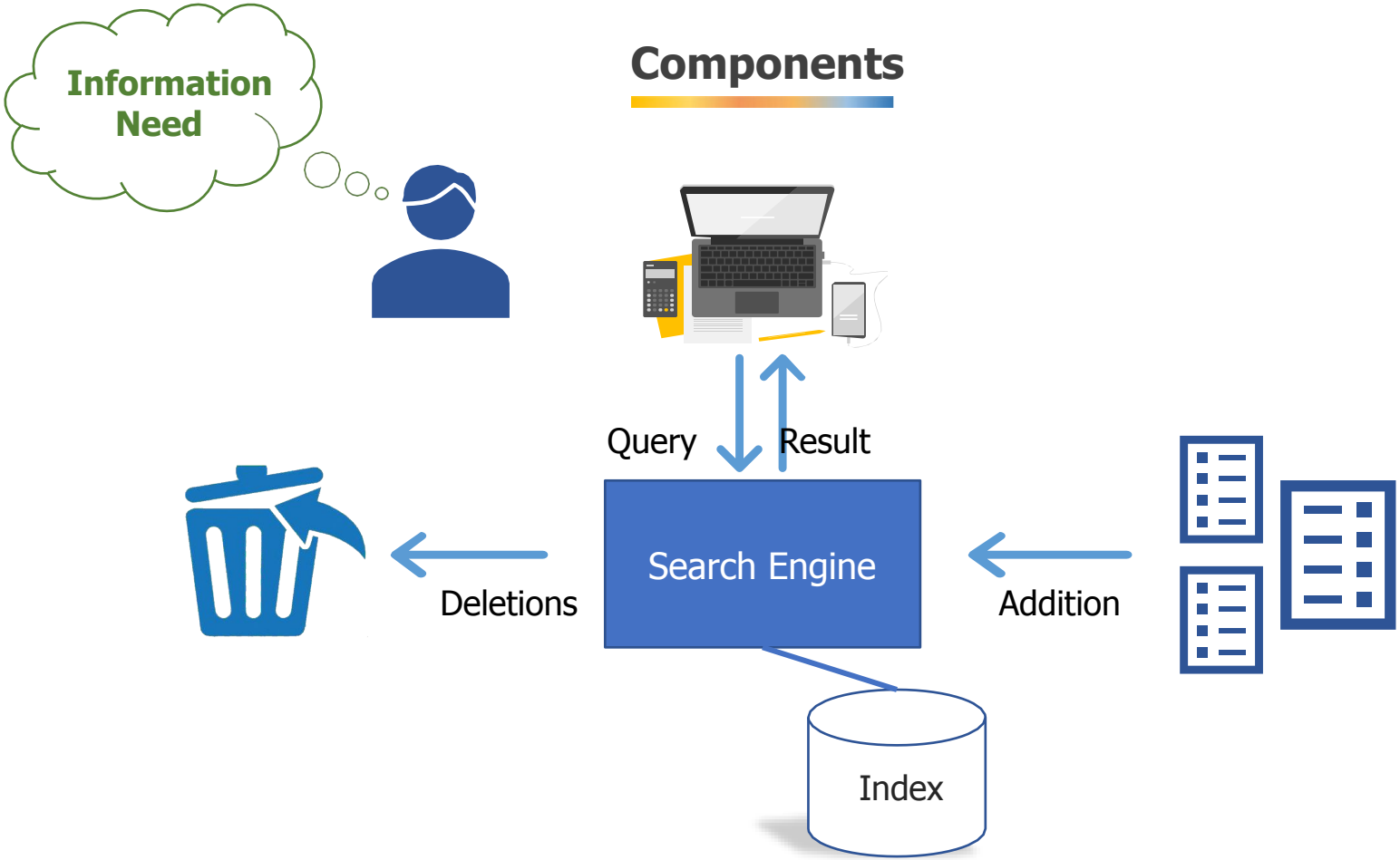
Corpus : Large repository of document stored in computer

Information Need: A topic about which we want to get information

Relevance: Some of the documents that may contain what I want to search

*“ So I express my need in the form of **Query** ”*

Components


Information Need
Query
Result
Search Engine
Deletions
Addition
Index

What is Term-Document Incidence Matrix

An example information retrieval problem

To understand problem of IR in general, consider our data "**Operating System Playlist"**

Now information that I want to retrieve is "*Which video in my playlist contains the terms: **Process** and **CPU** but not deadlock*"

Traditional Solution

- To perform sorting of Linear scan

- This process is commonly **GREP** referred to as grepping through text, after the **UNIX** command **grep**, which performs this process

- The way to avoid linear scanning the texts for each query is to index the documents in advance

Better Solution

- Processes the Corpus (all document) in advance
- For each video (document), record which term appear in it
- The result is a binary **Term – Document Incidence Matrix**

Example:	Process control block	Process Scheduling	CPU Utilization	Deadlock in operating system	Disk Scheduling algorithm	Critical section
Process	1	1	0	0	0	1
Kernel	0	1	0	1	0	0
CPU	1	1	1	0	0	0
Scheduling	0	1	0	0	1	0
deadlock	0	0	0	1	0	1

- Row represent: Distinct terms that appear in video
- **Column represent:** Difference video in operating System playlist **(Mark 1 for word appear in video otherwise 0)**

Example:	Process control block	Proc1ess Scheduling	CPU Utilization	Deadlock in operating system	Disk Scheduling algorithm	Critical section
Process	1	1	0	0	0	1
Kernel	0	1	0	1	0	0
CPU	1	1	1	0	0	0
Scheduling	0	1	0	0	1	0

- **Our Query: Process** AND **CPU** but not **deadlock**

  110001 AND **111000** AND **111010 (take complement)****

- **Take bitwise AND operation:** 110000
  - Answer: Process control block and process scheduling video contain word **process** and **CPU** but not deadlock

** Take 1 when 0, 0 when 1

What is Inverted Index

More realistic scenario

Suppose corpus has 1 million documents (index)

- Number of distinct term: 100,000

- Now if we create Term Document Incidence Matrix for this scenario

Number of cell in matrix: = 100,000 * 10,00,000

= 0.1 * 10^12 = 100GB

(if one cell is stored in 1 byte of memory)

Lots of memory space ....!!!

Building an Inverted Index

Token Sequencer
Sequence of each term in each document is given

Sorting
Sort the team alphabeticall y

Grouping
Group the term by words and then
document Id

Splitting into dictionary and posting
Dictionary store the terms and pointer to the posting list for each term.

Calculate Frequency
Document Frequency information is added

Build Inverted Index

Doc 1:
Ram an average student is studying but once he became friend of shyam he started attending lecture regularly.

Doc 2:
Shyam was very enthusiastic student. He had helped his friend Ram a lot. Now ram and shyam are good
friends

Building an inverted index - Example

Term	DocID
Ram	1
was	1
average	1
student	1
in	1
study	1
but	1
once	1
he	1
became	1
friend	1
of	1
Shyam	1
he	1
started	1
attending	1
lecture	1
regularly	1

Term	DocID
Shyam	2
was	2
very	2
enthusiastic	2
student	2
He	2
had	2
helped	2
his	2
friend	2
Ram	2
a	2
lot	2
Now	2
Ram	2
and	2
Shyam	2
are	2
good	2
friend	2

Term	DocID
a	2
and	2
are	2
attending	1
average	1
became	1
but	1
enthusiastic	2
friend	1
friend	2
friend	2
good	2
had	2
he	1
he	1
He	2
helped	2
his	2
in	1
in	1
lecture	1

Term	DocID
lecture	1
lot	2
lot	2
Now	2
Now	2
of	1
once	1
Ram	1
Ram	2
Ram	2
regularly	1
Shyam	1
Shyam	2
Shyam	2
started	1
student	1
student	2
study	1
very	2
was	1
was	2

Building an inverted index - Example

Term	DocID
a	2
and	2
are	2
attending	1
average	1
became	1
but	1
enthusiastic	2
friend	1
friend	2
friend	2
good	2
had	2
he	1
he	1
He	2
helped	2
his	2
in	1
lecture	1
lot	2

Term	DocID
Now	2
in	1
lecture	1
lot	2
Now	2
of	1
once	1
Ram	1
Ram	2
Ram	2
regularly	1
Shyam	1
Shyam	2
Shyam	2
started	1
student	1
student	2
study	1
very	2
was	1
was	2

Term	Doc. Frequency	Posting List	
a	1	2	
and	1	2	
are	1	2	
attending	1	1	
average	1	1	
became	1	1	
but	1	1	
enthusiastic	1	2	
friend	2	1	2
good	1	2	
had	1	1	
he	1	1	
He	1	2	
helped	1	2	
his	1	2	
in	1	1	
lecture	1	1	
lot	1	2	
Now	1	2	
of	1	1	
once	1	1	
Ram	2	1	2
regularly	1	1	
Shyam	2	1	2
started	1	1	
student	2	1	2
very	1	2	
was	2	1	2

What is Boolean Query Retrieval

In this model we represent query which is in Boolean expression of terms

Terms are combined with AND, NOR , NOT

X AND Y : represent doc that contains both X and Y

X OR Y: represent doc that contains either X or Y

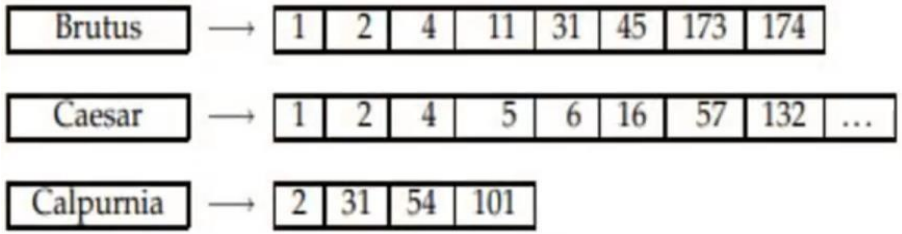
NOT X : represent the doc that do not contain X

Processing Boolean Queries

How do we process a query using an inverted index and the basic Boolean Retrieval model ?

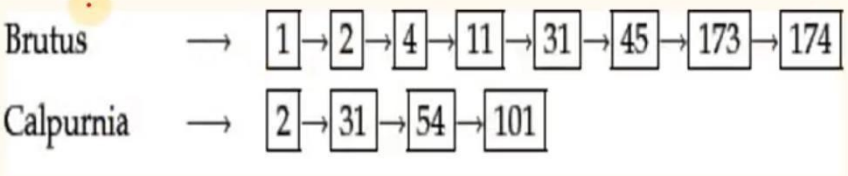
Brutus AND Calpurnia

1. Locate Brutus in the dictionary
2. Retrieve its posting
3. Locate Calpurnia in the Dictionary
4. Retrieve its posting
5. Intersect the two posting list



Implementation(Brutus AND Calpurnia)

```
INTERSECT(p1, p2)
  answer ← ⟨ ⟩
  while p1 ≠ NIL and p2 ≠ NIL
  do if docID(p1) = docID(p2)
        then ADD(answer, docID(p1))
             p1 ← next(p1)
             p2 ← next(p2)
        else if docID(p1) < docID(p2)
                then p1 ← next(p1)
                else p2 ← next(p2)
  return answer
```



2	31			

Answer : based on Intersection

Implementation(Brutus OR Calpurnia)

Brutus ⟶ 1 → 2 → 4 → 11 → 31 → 45 → 173 → 174

Calpurnia ⟶ 2 → 31 → 54 → 101

1	2	4	11	31	45	54	101	173. .

Answer : based on Union

Implementation(NOT Brutus)

Brutus ⟶ [1] → [2] → [4]

3	5	6	7	8	9	10		

Answer : based on NOT

Indexing Process (4 Steps)

1 TOKENISATION

2 STOPWORD ELIMNATION

3 STEMMING

4 CREATION OF THE INVERTED INDEX

Step:1 Tokenization

- **Tokenisation**: First of all we need to extract all **terms** in a document (we do this for all documents in the collection)
- A term might be a single word, but could also be a phrase. *For example, 'computer science' could be one term, or split into the two terms 'computer' and 'science'*
- We may not only consider the occurrence of a term, but also its position in the document
- Useful for query term highlighting or phrase and proximity search
- This builds our **vocabulary** (the set of all different terms found in all documents in the collection)

What is Tokenization

Tokenization is the task of chopping up text into pieces, called Tokens. Perhaps at the same time throwing away certain characters.

Input: Krishna, Happy, Learning, Lifelong process

Output: Krishna | Happy | Learning | Lifelong | process

These tokens are often loosely referred to as **terms** or **words,** but it is sometimes important to make a type/token distinction

Token Vs Type Vs Term

Token	Type	Term
A token is an instance of a sequence of characters in some particular document that are grouped together as a useful sematic unit for processing	A type is the class of all tokens containing the same character sequence.	A Term is a (perhaps normalized) type that is included in the IR system's dictionary.

Token Vs Type Vs Term: example

Document	Are you able to login to system
Token	Are, you, able, to, login , to system
Type	Are you able to login system
Term	Are you able login system

Step:2 Stopword Elimination

- Frequent terms like “and”, “or” don’t bear any meaning for ad hoc retrieval
- This means we can eliminate these terms to keep our vocabulary small (usually stopword lists are applied)

- Example
  - Apple has a built-in RSS reader. Kinda. While it’s not the dedicated RSS client that I hope they’ll build for iPhone, it helps a news junkie get his fix on a slow EDGE data connection.
  - After stopword elimination (and converting terms to lower case): *apple built-in rss reader kinda dedicated rss client hope build iPhone helps news junkie get fix slow edge data connection*

Step:3 Stemming

- **Stemming** is the process of reducing terms to their **stem base**
- More potentially relevant documents can be found
- Stemming method for the English language: **Porter algorithm**
- Several types of stemmers for different languages exist (for instance dictionary-based)

- *Further Resources*
  - *Online stemmer demo :* *http://www.9ol.es/porter_js_demo.html*
  - *Porter Stemmer Page:* *http://tartarus.org/~martin/PorterStemmer/*

Stemming Example

Example:

- Computer → comput
- Computers → comput
- Computing → comput

  → Search for 'computer' also finds documents with 'computers' or 'computing' (if we

  stem both the document and query terms)

- The previous text example after stopword elimination and stemming:

  appl built rss read kinda dedic rss cli hop build iphon help new junki get fix slow

  edg data connect

Step:4 Inverted Index and Inverted List

- After tokenisation, stemming and stopword elimination (to create our vocabulary) we can build the inverted index
  - Ensures fast access to documents containing a term, similar to an index in a book
- The **inverted index** consists of inverted lists
- **Inverted list**: One of the basic data structures in IR
- Term-oriented
- There is an inverted list for each term in the vocabulary
- Each such list stores
  - The identifiers (IDs, e.g. URLs) of the documents that contain that term
  - All its positions in the document (if needed)
  - Further information as required
- We can now efficiently process queries and retrieve documents with the inverted index

Issue with Tokenization

The major question of the Tokenization phase is what are the **correct token to use** ?

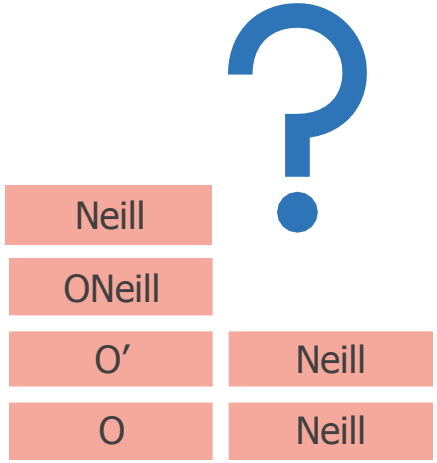
It look fairly trivial: you chop on whitespace and throw away punctuation character

Example: Apostrophe

Document

Mr. O'Neill thinks that the boy's stories about Chile's capital aren't amazing

What should
be
for aren't ?



Co-education
?
Long-Term ?
Lowercase ?

Language Issue

German writes compound nouns without spaces e.g., Computerlinguistik “Computer Linguistic”

Compound Splitter module which is usually implemented by seeing if a word can be subdivided into multiple words that appear in a vocabulary

East Asian Language (E.g., Chinese, Japanese, Korean and Thai) where text is written with any spaces between words.

Boolean Retrieval Model

Also known as exact – match retrieval

Assumptions: All documents in the retrieved set are equivalent in terms of relevance

Two possible outcomes for every evaluation (TRUE & FALSE)

Query is usually specified using operators for Boolean logic (AND, OR, NOT)

Example Boolean Query

Query: Lincoln
Retrieve a large amount of documents that mention Lincoln cars and place named Lincoln in addition to stories about President Lincoln.

Query: President AND Lincoln:
Retrieve a set of documents that contain both words, occurring anywhere in the documents

Query: President AND Lincoln AND NOT (Automobile or car) Removes too many relevant documents along with non-relevant

Query: President AND Lincoln AND Biography AND life AND Birthplace AND Gettysburg and NOT (automobile OR Car)
Putting too many search term into the query with the AND operator often result in nothing being received.

Example Search : AND

Literature review on

"Retrieval systems for multimedia objects, in particular images"

- We need a suitable query!
- Possible query:

  retrieval system AND multimedia AND images
- Interpretation in a Boolean retrieval system:

  Return all documents in which **all** of these terms appear at least once
- Example of a document returned:

  "'**Multimedia** information **retrieval system** using knowledge for recognition of

  images"
- Note "retrieval system" is regarded as one term in this example

Example Search : OR

Literature review on

"Retrieval systems for multimedia objects, in particular images"

- Another Possible query:

  retrieval system OR multimedia OR images

- Interpretation in a Boolean retrieval system:

  Return all documents in which **any** of these terms appear at least once

- Example of a document returned:

  "Recognition of **images** in medical applications"

Effect of AND/OR

Query	#Docs retrieved
`retrieval systems AND multimedia AND images`	3
`retrieval AND multimedia`	148
`retrieval OR multimedia`	2559

- AND is a strict filter (less documents found)
- OR is less strict (many documents found)

Vector Space Model

In this model, documents and queries are assumed to be a part of a **t-dimensional** vector space, where **t**
is the number of index terms (**Word, Stems. Phrases** etc.)

A document ***Di*** is represented by a vector of index terms:

Di = (di1, di2,……dit).

Where ***Dij*** represent the weight of the Jth term *i.e. no. of time the word appear in the document*.

Query Q is represented by a vector of ***t*** weight:
Q= (q1,q2….qT) i.e.

Vector Space Model

A document collection containing n documents can be represented as a matrix of term weights.

Column – term weight

Row – document

$$
\begin{array}{cccccc}
 & Term_1 & Term_2 & \dots & Term_t \\
Doc_1 & d_{11} & d_{12} & \dots & d_{1t} \\
Doc_2 & d_{21} & d_{22} & \dots & d_{2t} \\
\vdots & \vdots & & & \\
Doc_n & d_{n1} & d_{n2} & \dots & d_{nt}
\end{array}
$$

- The **term weights** are simply the count of the terms in the document.
- **Stopwords** are not indexed in this example
- The word have been **stemmed**

Vector Space Model

D_1 Tropical Freshwater Aquarium Fish.
D_2 Tropical Fish, Aquarium Care, Tank Setup.
D_3 Keeping Tropical Fish and Goldfish in Aquariums, and Fish Bowls.
D_4 The Tropical Tank Homepage - Tropical Fish and Aquariums.

Terms	Documents			
	D_1	D_2	D_3	D_4
aquarium	1	1	1	1
bowl	0	0	1	0
care	0	1	0	0
fish	1	1	2	1
freshwater	1	0	0	0
goldfish	0	0	1	0
homepage	0	0	0	1
keep	0	0	1	0
setup	0	1	0	0
tank	0	1	0	1
tropical	1	1	1	2

Document D3, for example is represented buy the vector(1,1,0,2,0,1,0,1,0,0,1)
t=11 (11 dimensional i.e. unique terms)

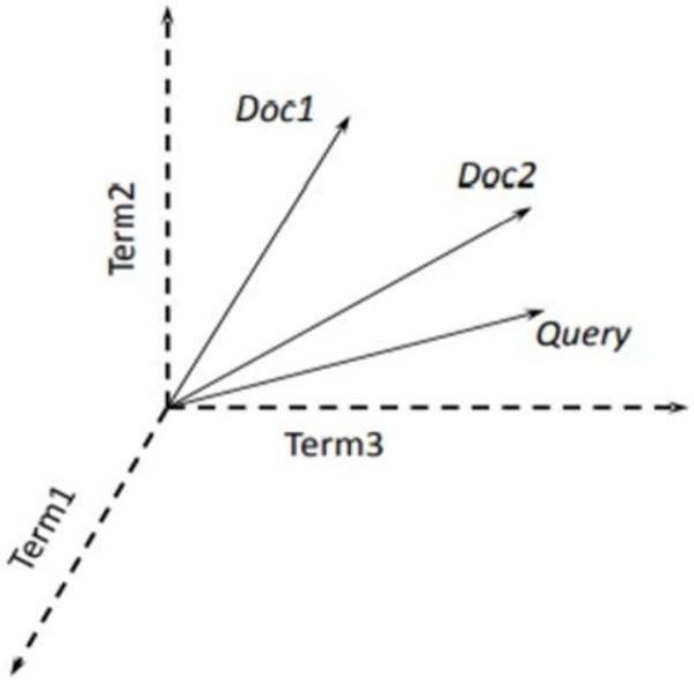
Query: "Tropical fish", vector representation of query be (0,0,0,1,0,0,0,0,0,0,0, 1)

Vector Space Model

Given this representation, documents could be ranked by computing the distance between the points representing the documents and the query

A similarity measure is used (rather than a distance or dissimilarity measure)

The documents with the highest scores are the most similar to the query.



Vector Space Model

The cosine correlation measures the cosine of the angle between the query and the document vectors.

The cosine measure is defined as

$$Cosine(D_i, Q) = \frac{\sum_{j=1}^{t} d_{ij} \cdot q_j}{\sqrt{\sum_{j=1}^{t} {d_{ij}}^2 \cdot \sum_{j=1}^{t} {q_j}^2}}$$

Vector Space Model

As an example, consider two documents D1 = (0.5, 0.8, 0.3) and D2 = (0.9, 0.4, 0.2) indexed by three terms, where the numbers represent term **weights**.

Given the query Q = (1.5, 1.0, 0) indexed by the same terms, the cosine measures for the two documents are:

$$Cosine(D_1, Q) = \frac{(0.5 \times 1.5) + (0.8 \times 1.0)}{\sqrt{(0.5^2 + 0.8^2 + 0.3^2)(1.5^2 + 1.0^2)}} = \frac{1.55}{\sqrt{(0.98 \times 3.25)}} = 0.87$$

$$Cosine(D_2, Q) = \frac{(0.9 \times 1.5) + (0.4 \times 1.0)}{\sqrt{(0.9^2 + 0.4^2 + 0.2^2)(1.5^2 + 1.0^2)}} = \frac{1.75}{\sqrt{(1.01 \times 3.25)}} = 0.97$$